

Тема: Подключение к PostgreSQL через database/sql. Выполнение простых запросов (INSERT, SELECT)

Цели:

1. Установить и настроить PostgreSQL локально.
2. Подключиться к БД из Go с помощью database/sql и драйвера PostgreSQL.
3. Выполнить параметризованные запросы INSERT и SELECT.
4. Корректно работать с context, пулом соединений и обработкой ошибок.

1. Ход работы

1.1. Установка и проверка PostgreSQL

Установил PostgreSQL 16 с сайта EnterpriseDB, задал пароль суперпользователя postgres. Убедился, что служба запущена:

```
PS C:\Users\Falcion> go version
go version go1.27.1 windows/amd64
PS C:\Users\Falcion> Get-Service postgresql*

Status      Name                DisplayName
-----
Running     postgresql-x64-16  postgresql-x64-16 - PostgreSQL Serv...

PS C:\Users\Falcion> |
```

Подключился к серверу (PowerShell):

```
Пароль пользователя Falcion:

psql: ошибка: подключиться к серверу "localhost" (:::1), порту 5432 не удалось: Permission denied (0x0000271D/10013)
Сервер действительно работает по данному адресу и принимает TCP-соединения?
подключиться к серверу "localhost" (127.0.0.1), порту 5432 не удалось: ВАЖНО: пользователь "Falcion" не прошёл проверку
подлинности (по паролю)
PS C:\Users\Falcion> psql -U postgres
Пароль пользователя postgres:

psql (16.3)
ПРЕДУПРЕЖДЕНИЕ: Кодовая страница консоли (866) отличается от основной
страницы Windows (1251).
8-битовые (русские) символы могут отображаться некорректно.
Подробнее об этом смотрите документацию psql, раздел
"Notes for Windows users".
Введите "help", чтобы получить справку.

postgres=# \q
PS C:\Users\Falcion> go version
go version go1.27.1 windows/amd64
PS C:\Users\Falcion> |
```

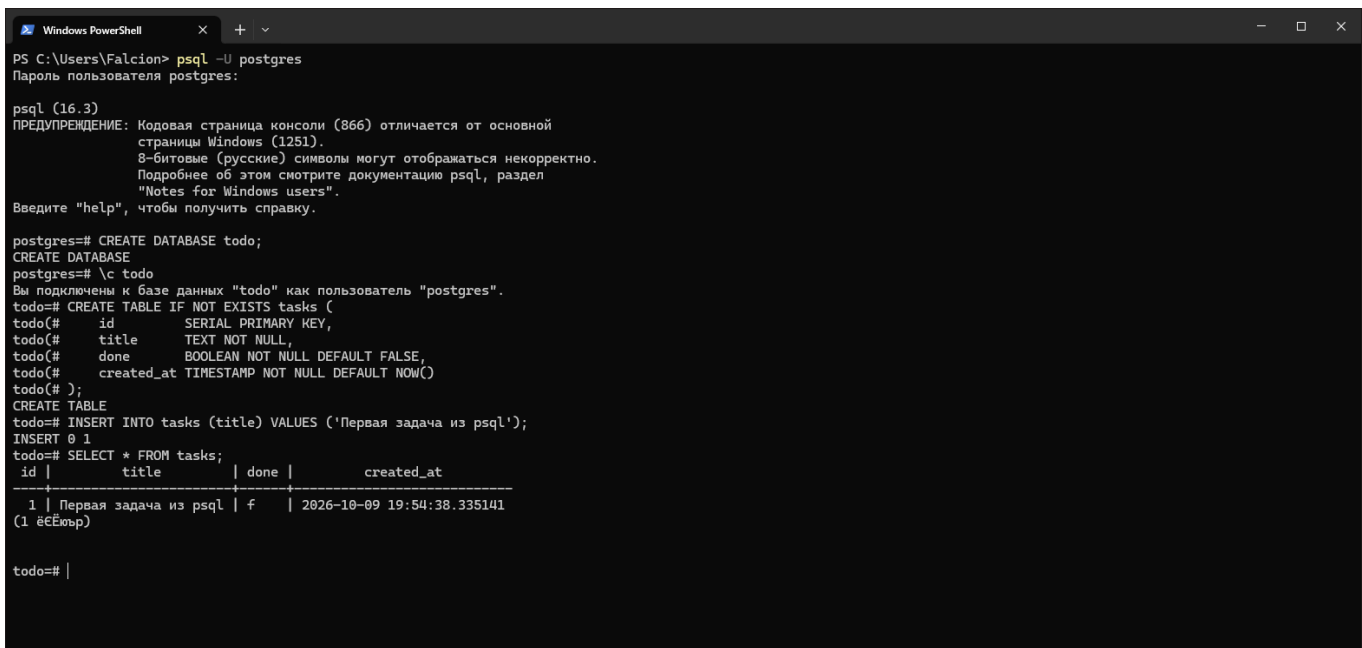
1.2. Создание БД и таблицы

Выполним:

```
CREATE DATABASE todo;
\c todo

CREATE TABLE IF NOT EXISTS tasks (
    id          SERIAL PRIMARY KEY,
    title       TEXT NOT NULL,
    done        BOOLEAN NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- добавим одну строку вручную, чтобы проверить работу из psql
INSERT INTO tasks (title) VALUES ('Первая задача из psql');
SELECT * FROM tasks;
```



```
Windows PowerShell
PS C:\Users\Falcion> psql -U postgres
Пароль пользователя postgres:

psql (16.3)
ПРЕДУПРЕЖДЕНИЕ: Кодовая страница консоли (866) отличается от основной
страницы Windows (1251).
8-битовые (русские) символы могут отображаться некорректно.
Подробнее об этом смотрите документацию psql, раздел
"Notes for Windows users".
Введите "help", чтобы получить справку.

postgres=# CREATE DATABASE todo;
CREATE DATABASE
postgres=# \c todo
Вы подключены к базе данных "todo" как пользователь "postgres".
todo=# CREATE TABLE IF NOT EXISTS tasks (
todo(#   id          SERIAL PRIMARY KEY,
todo(#   title       TEXT NOT NULL,
todo(#   done        BOOLEAN NOT NULL DEFAULT FALSE,
todo(#   created_at  TIMESTAMP NOT NULL DEFAULT NOW()
todo(# );
CREATE TABLE
todo=# INSERT INTO tasks (title) VALUES ('Первая задача из psql');
INSERT 0 1
todo=# SELECT * FROM tasks;
 id | title | done | created_at
---+-----+-----+-----
  1 | Первая задача из psql | f    | 2026-10-09 19:54:38.335141
(1 ёсЁмър)

todo=# |
```

1.3. Создание Go-проекта

```
mkdir pz5-postgres
cd pz5-postgres
go mod init pz5-postgres
go get github.com/jackc/pgx/v5/stdlib
go get github.com/joho/godotenv
```

Попробуйте новую кроссплатформенную оболочку PowerShell (<https://aka.ms/pscore6>)
PS D:\.WORKSPACE\Education\T1.5> **mkdir** pz5-postgres

Каталог: D:\.WORKSPACE\Education\T1.5

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	09.10.2026	19:55	pz5-postgres

```
PS D:\.WORKSPACE\Education\T1.5> cd pz5-postgres
PS D:\.WORKSPACE\Education\T1.5\pz5-postgres> go mod init pz5-postgres
go: creating new go.mod: module pz5-postgres
PS D:\.WORKSPACE\Education\T1.5\pz5-postgres> go get github.com/jackc/pgx/v5/stdlib
go: downloading github.com/jackc/pgx v3.6.2+incompatible
go: downloading github.com/jackc/pgx/v5 v5.11.0
go: downloading github.com/jackc/pgservicefile v0.0.0-20240606120523-5a60cdf6a761
go: downloading golang.org/x/text v0.29.0
go: downloading github.com/jackc/puddle/v2 v2.2.2
go: downloading github.com/jackc/pgpassfile v1.0.0
go: downloading golang.org/x/sync v0.17.0
go: added github.com/jackc/pgpassfile v1.0.0
go: added github.com/jackc/pgservicefile v0.0.0-20240606120523-5a60cdf6a761
go: added github.com/jackc/pgx/v5 v5.11.0
go: added github.com/jackc/puddle/v2 v2.2.2
go: added golang.org/x/sync v0.17.0
go: added golang.org/x/text v0.29.0
PS D:\.WORKSPACE\Education\T1.5\pz5-postgres> go get github.com/joho/godotenv
go: downloading github.com/joho/godotenv v1.5.1
go: added github.com/joho/godotenv v1.5.1
PS D:\.WORKSPACE\Education\T1.5\pz5-postgres> |
```

Структура проекта:

```
pz5-postgres/
├─ go.mod
├─ go.sum
├─ .env
├─ db.go
├─ repository.go
└─ main.go
```

Файл .env :

```
DATABASE_URL=postgres://postgres:admin@localhost:5432/todo?sslmode=disable
```

1.4. Код проекта

1.4.1. db.go

```
package main

import (
    "context"
    "database/sql"
    "log"
    "time"

    _ "github.com/jackc/pgx/v5/stdlib"
)

func openDB(dsn string) (*sql.DB, error) {
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        return nil, err
    }

    // Настройки пула соединений
    db.SetMaxOpenConns(10)
    db.SetMaxIdleConns(5)
    db.SetConnMaxLifetime(30 * time.Minute)

    // Проверка соединения с таймаутом
    ctx, cancel := context.WithTimeout(context.Background(),
2*time.Second)
    defer cancel()
    if err := db.PingContext(ctx); err != nil {
        return nil, err
    }

    log.Println("Connected to PostgreSQL")
    return db, nil
}
```

1.4.2. repository.go

```
package main

import (
    "context"
```

```

        "database/sql"
        "time"
    )

// Task – модель для сканирования результатов SELECT
type Task struct {
    ID        int
    Title     string
    Done      bool
    CreatedAt time.Time
}

type Repo struct {
    DB *sql.DB
}

func NewRepo(db *sql.DB) *Repo {
    return &Repo{DB: db}
}

// CreateTask – параметризованный INSERT с возвратом id
func (r *Repo) CreateTask(ctx context.Context, title string) (int, error) {
    var id int
    const q = `INSERT INTO tasks (title) VALUES ($1) RETURNING id;`
    err := r.DB.QueryRowContext(ctx, q, title).Scan(&id)
    return id, err
}

// ListTasks – базовый SELECT всех задач
func (r *Repo) ListTasks(ctx context.Context) ([]Task, error) {
    const q = `SELECT id, title, done, created_at FROM tasks ORDER BY id;`
    rows, err := r.DB.QueryContext(ctx, q)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var out []Task
    for rows.Next() {
        var t Task
        if err := rows.Scan(&t.ID, &t.Title, &t.Done, &t.CreatedAt);
err != nil {
            return nil, err
        }
        out = append(out, t)
    }
}

```

```

        return out, rows.Err()
    }

// ----- Задание 1: ListDone -----
func (r *Repo) ListDone(ctx context.Context, done bool) ([]Task, error) {
    const q = `SELECT id, title, done, created_at FROM tasks WHERE done =
$1 ORDER BY id;`
    rows, err := r.DB.QueryContext(ctx, q, done)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var out []Task
    for rows.Next() {
        var t Task
        if err := rows.Scan(&t.ID, &t.Title, &t.Done, &t.CreatedAt);
err != nil {
            return nil, err
        }
        out = append(out, t)
    }
    return out, rows.Err()
}

// ----- Задание 2: FindByID -----
func (r *Repo) FindByID(ctx context.Context, id int) (*Task, error) {
    const q = `SELECT id, title, done, created_at FROM tasks WHERE id =
$1;`
    var t Task
    err := r.DB.QueryRowContext(ctx, q, id).Scan(&t.ID, &t.Title, &t.Done,
&t.CreatedAt)
    if err != nil {
        return nil, err
    }
    return &t, nil
}

// ----- Задание 3: CreateMany (через транзакцию) -----
func (r *Repo) CreateMany(ctx context.Context, titles []string) error {
    tx, err := r.DB.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    // Откат при любой ошибке (в т.ч. панике) до Commit
    defer tx.Rollback()

```

```

const q = `INSERT INTO tasks (title) VALUES ($1);`
stmt, err := tx.PrepareContext(ctx, q)
if err != nil {
    return err
}
defer stmt.Close()

for _, title := range titles {
    if _, err := stmt.ExecContext(ctx, title); err != nil {
        return err
    }
}
return tx.Commit()
}

```

1.4.3. main.go

```

package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "time"

    "github.com/joho/godotenv"
)

func main() {
    // .env не обязателен; если файла нет – ошибка игнорируется
    _ = godotenv.Load()
    dsn := os.Getenv("DATABASE_URL")
    if dsn == "" {
        // fallback только для учебного стенда!
        dsn = "postgres://postgres:YOUR_PASSWORD@localhost:5432/todo?sslmode=disable"
    }

    db, err := openDB(dsn)
    if err != nil {
        log.Fatalf("openDB error: %v", err)
    }
    defer db.Close()

```

```

repo := NewRepo(db)

ctx, cancel := context.WithTimeout(context.Background(),
5*time.Second)
defer cancel()

// 1) INSERT – по одной задаче
titles := []string{"Сделать ПЗ №5", "Купить кофе", "Проверить отчёты"}
for _, title := range titles {
    id, err := repo.CreateTask(ctx, title)
    if err != nil {
        log.Fatalf("CreateTask error: %v", err)
    }
    log.Printf("Inserted task id=%d (%s)", id, title)
}

// 2) SELECT всех задач
tasks, err := repo.ListTasks(ctx)
if err != nil {
    log.Fatalf("ListTasks error: %v", err)
}
fmt.Println("=== Tasks ===")
for _, t := range tasks {
    fmt.Printf("%d | %-24s | done=%-5v | %s\n",
        t.ID, t.Title, t.Done,
t.CreatedAt.Format(time.RFC3339))
}

// 3) Задание 1: ListDone
notDone, err := repo.ListDone(ctx, false)
if err != nil {
    log.Fatalf("ListDone error: %v", err)
}
fmt.Println("\n=== ListDone(false) ===")
for _, t := range notDone {
    fmt.Printf("%d | %s | done=%v\n", t.ID, t.Title, t.Done)
}

// 4) Задание 2: FindByID
if len(tasks) > 0 {
    t, err := repo.FindByID(ctx, tasks[0].ID)
    if err != nil {
        log.Fatalf("FindByID error: %v", err)
    }
    fmt.Printf("\n=== FindByID(%d) ===\n", t.ID)
}

```

```

        fmt.Printf("ID=%d Title=%q Done=%v CreatedAt=%s\n",
            t.ID, t.Title, t.Done,
            t.CreatedAt.Format(time.RFC3339))
    }

    // 5) Задание 3: CreateMany
    batch := []string{"Задача A", "Задача B", "Задача C"}
    if err := repo.CreateMany(ctx, batch); err != nil {
        log.Fatalf("CreateMany error: %v", err)
    }
    log.Printf("CreateMany inserted %d tasks", len(batch))
}

```

1.5. Запуск приложения

Запустим проект:

```
go run .
```

```

exit status 1
PS D:\WORKSPACE\Education\T1.5\pz5-postgres> go run .
2026/10/09 19:59:52 Connected to PostgreSQL
2026/10/09 19:59:52 Inserted task id=2 (Сделать ПЗ №5)
2026/10/09 19:59:52 Inserted task id=3 (Купить кофе)
2026/10/09 19:59:52 Inserted task id=4 (Проверить отчёты)
=== Tasks ===
1 | ЫГаў п § № з Ё§ psql | done=false | 2026-10-09T19:54:38Z
2 | Сделать ПЗ №5 | done=false | 2026-10-09T19:59:52Z
3 | Купить кофе | done=false | 2026-10-09T19:59:52Z
4 | Проверить отчёты | done=false | 2026-10-09T19:59:52Z

=== ListDone(false) ===
1 | ЫГаў п § № з Ё§ psql | done=false
2 | Сделать ПЗ №5 | done=false
3 | Купить кофе | done=false
4 | Проверить отчёты | done=false

=== FindByID(1) ===
ID=1 Title="ЫГаў\u00a0п §\u00a0№\u00a0з\u00a0 Ё§ psql" Done=false CreatedAt=2026-10-09T19:54:38Z
2026/10/09 19:59:52 CreateMany inserted 3 tasks
PS D:\WORKSPACE\Education\T1.5\pz5-postgres> |

```

Проверка через psql:

```

INSERT 0 1
todo=# SELECT * FROM tasks;
 id |          title          | done |          created_at
-----+-----+-----+-----
  1 | Первая задача из psql | f    | 2026-10-09 19:54:38.335141
(1 ÷)

todo=# SELECT * FROM tasks ORDER BY id;
 id |          title          | done |          created_at
-----+-----+-----+-----
  1 | Первая задача из psql | f    | 2026-10-09 19:54:38.335141
  2 | 2-я задача из psql    | f    | 2026-10-09 19:59:52.373401
  3 | 3-я задача из psql    | f    | 2026-10-09 19:59:52.39806
  4 | 4-я задача из psql    | f    | 2026-10-09 19:59:52.398905
  5 | 5-я задача из psql    | f    | 2026-10-09 19:59:52.408079
  6 | 6-я задача из psql    | f    | 2026-10-09 19:59:52.408079
  7 | 7-я задача из psql    | f    | 2026-10-09 19:59:52.408079
(7 ÷)

```

И через новое подключение/клиент:

Попробуйте новую кроссплатформенную оболочку PowerShell (<https://aka.ms/pscore6>)

```

PS C:\Users\Falcion> psql -U postgres -d todo -c "SELECT id, title, done, created_at FROM tasks ORDER BY id;"
Пароль пользователя postgres:
 id |          title          | done |          created_at
-----+-----+-----+-----
  1 | Первая задача из psql | f    | 2026-10-09 19:54:38.335141
  2 | 2-я задача из psql    | f    | 2026-10-09 19:59:52.373401
  3 | 3-я задача из psql    | f    | 2026-10-09 19:59:52.39806
  4 | 4-я задача из psql    | f    | 2026-10-09 19:59:52.398905
  5 | 5-я задача из psql    | f    | 2026-10-09 19:59:52.408079
  6 | 6-я задача из psql    | f    | 2026-10-09 19:59:52.408079
  7 | 7-я задача из psql    | f    | 2026-10-09 19:59:52.408079
(7 ÷)

```